



TP3

Perceptrones

TP3 — Perceptrones
Resultados experimentales sobre tres ejercicios

Sistemas de Inteligencia Artificial · ITBA · 1Q 2026

Bloque 1 / 3

Ej 1 — Knowledge Distillation

Ej 1 — El problema

Knowledge distillation: replicar la salida del BigModel pre-entrenado con un TinyModel (perceptrón simple).

Datos: `fraud_dataset.csv`

9 features (amount, session, account_age, ...)

Target de entrenamiento:

`big_model_fraud_probability` (continuo en $[0, 1]$).

Criterio de evaluación:

`flagged_fraud` (binaria 0/1).

Regla del enunciado:

`flagged_fraud` *nunca* se usa para entrenar. Solo para métricas operativas (precision, recall, F1) post-hoc.

Salida esperada en $[0, 1]$ $\Rightarrow$
activación sigmoide.

Ej 1 — Análisis del dataset: tres reglas determinísticas

Antes de entrenar: la exploración del dataset reveló **umbrales duros** en tres features que separan las clases.

```
amount_usd > 500 ∨ quantity_purchased ≥ 10 ∨ items_viewed_before_purchase ≥ 15
```

Aporte individual:

Regla	Matches	TP	FP
<code>amount_usd > 500</code>	199	199	0
<code>quantity ≥ 10</code>	524	524	0
<code>items_viewed ≥ 15</code>	490	490	0

Las tres juntas marcan **695** filas únicas.

OR como predictor (`simple_prediction`):

	Predicción	
	1 (fraude)	0 (no fraude)
Real 1 (fraude)	TP=695	FN=174
Real 0 (no fraude)	FP=0	TN=6631

Accuracy	97.68 %
Precision	100.00 %
Recall	79.98 %

Ej 1 — Conclusión del análisis del dataset

El 80 % del fraude se etiqueta correctamente con tres *if*, sin un solo falso positivo.

El 20 % restante (174 casos) es el verdadero desafío del perceptrón: son fraudes que respetan los tres umbrales y solo se detectan combinando features continuas.

Implicancias para el TinyModel:

- **Línea de base:** el modelo debe igualar a las tres reglas ($\text{recall} \geq 80\%$).
- **Valor agregado:** se mide en cuántos de los 174 fraudes “sutiles” logra recuperar.
- **Lineal vs no-lineal:** las discontinuidades duras (saltos $\sim 6\% \rightarrow 100\%$) son imposibles para una recta — el perceptrón no-lineal debe mostrar ventaja.

Ej 1 — Lineal vs No-Lineal: MSE de distillation

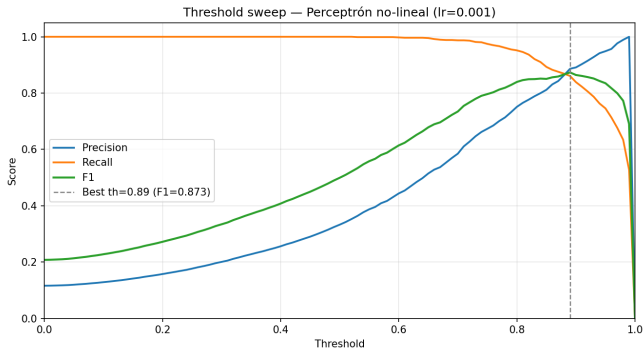
Mismo dataset, K-fold = 5 estratificado, mismas features. Diferencia: presencia de la sigmoide.

Modelo	MSE test (mean $\pm$ std)
Lineal (Adaline)	0,0266 $\pm$ 0,0008
No-Lineal (sigmoide)	0,0110 $\pm$ 0,0004 (−59 %)

¿Por qué la sigmoide gana tan claro?

El target es una probabilidad acotada en $[0, 1]$. La sigmoide *naturalmente* mapea su salida a ese rango. El lineal puede producir valores fuera de $[0, 1]$ y el MSE penaliza fuerte esa salida fuera de dominio.

Ej 1 — Pruebas sobre threshold



Óptimo por F1: threshold = 0.89.

Precision 0,886, recall 0,861, F1 0,873, accuracy 0,971.

Ej 1 — Recomendación de umbral

La elección depende del costo asimétrico falso positivo / falso negativo.

Threshold	Precision	Recall	F1	FP
0.50 (default)	0.333	1.000	0.499	1743
0.89 (óptimo F1)	0.886	0.861	0.873	96
0.95	0.949	0.746	0.835	35
0.99	1.000	0.527	0.690	0

Bloque 2 / 3

Ej 2 — Clasificación de dígitos

Ej 2 — El problema

Clasificación multiclase de dígitos manuscritos (0–9).

Datos:

- Train: `digits.csv` (12.449 muestras)
- Test: `digits_test.csv` (2.498)

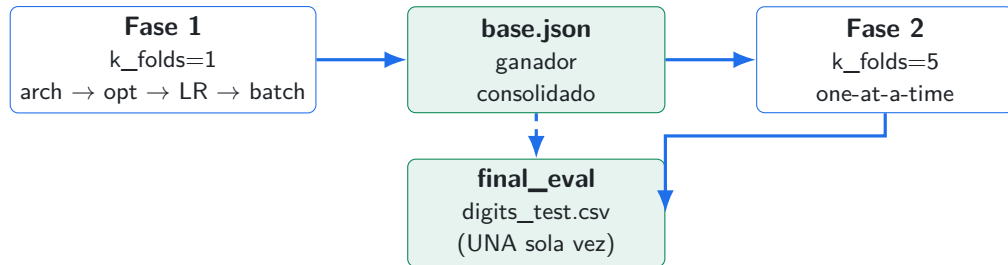
Cada muestra: vector de 784 floats $\in [0, 1]$
(28×28 flatten).

Arquitectura final:

- Input: 784
- Hidden 1: 100 (ReLU)
- Hidden 2: 50 (ReLU)
- Output: 10 (Softmax)

Regla: el test set se evalúa una sola vez al final.
Búsqueda de hiperparámetros sólo sobre `digits.csv`.

Ej 2 — Fases de experimentación



Fase 1: sweeps secuenciales para reducir el espacio de búsqueda.

Fase 2: K-fold=5 para validar la elección con comparativas robustas.

Ej 2 — Sweep de arquitectura

Arquitectura	val_acc	Comentario
[784, 100, 50, 10]	96.74 % / 96.22 %	Ganador por parsimonia
[784, 128, 64, 10]	96.86 % / —	Empate técnico ($\Delta < 0,4 \%$)
[784, 200, 100, 10] (<i>wider</i>)	— / 96.39 %	−0.48 pp test , 3,6× más lento
[784, 100, 50, 25, 10] (<i>deeper</i>)	— / 96.20 %	Empate
[784, 30, 10] (<i>shallow</i>)	— / 95.15 %	Subajustado

Decisión por parsimonia (regla de Occam): cuando dos arquitecturas rinden igual dentro del desvío, gana la más simple/rápida. **Wider** confirma empíricamente la curva U del PDF de regularización: aumentar capacidad sin más datos $\Rightarrow$ overfitting (mejor val, peor test).

Ej 2 — Sweep de optimizador

Optimizador	val_acc	best_ep	Comentario
Adam ($\alpha = 10^{-3}$)	96.74 %	7	Ganador
Momentum ($\beta = 0,9$)	96.34 %	18	Converge más lento
SGD vanilla	94.81 %	48	No convergió en 50 epochs

Adam gana por convergencia más rápida y resultado igual o mejor. El PDF de optimizadores lo califica textual como *“muy usado en la práctica”* (defaults idénticos al paper Kingma & Ba 2015). Momentum queda dentro del desvío; SGD claramente atrás sin aceleración.

Función de activación — cuál y por qué

Dónde	Cuál	Por qué
Ej 1 — distillation	sigmoide	Target en $[0, 1]$ (probabilidad). La sigmoide naturalmente devuelve ese rango.
Ej 2 / Ej 3 — capas ocultas	ReLU	Gradiente fuerte y constante en zona positiva. No satura como tanh/sigmoide. Entrena rápido.
Ej 2 / Ej 3 — capa de salida	softmax	Clasificación multiclase: devuelve distribución de probabilidad sobre las 10 clases.

¿Qué es “saturar”? Una activación satura cuando la pendiente $f'(z)$ se aproxima a 0 para entradas grandes en magnitud. Sigmoide y tanh saturan en sus extremos: el gradiente que llega a las capas previas se desvanece (*vanishing gradient*) y la red deja de aprender.

ReLU = $\max(0, z)$: derivada 1 en zona positiva, 0 en negativa. Sin saturación en su lado activo $\Rightarrow$ el gradiente fluye sin atenuarse a través de muchas capas. Es la activación estándar moderna para capas ocultas.

Inicialización de pesos — cuál y por qué

- **¿Por qué no cero?** Si todas las neuronas arrancan iguales, hacen exactamente lo mismo en cada capa. El gradiente las mueve igual y *nunca se diferencian* (problema de simetría).
- **¿Por qué pequeños?** Pesos grandes saturan las activaciones (especialmente sigmoide/tanh): la salida queda en zonas planas, el gradiente es casi cero, la red no aprende.

Esquema	Distribución	Cuándo lo usamos
He	$\mathcal{N}\left(0, \sqrt{2/\text{fan_in}}\right)$	Capas con ReLU : compensa que ReLU descarta la mitad del input.
Xavier	$\mathcal{N}\left(0, \sqrt{1/\text{fan_in}}\right)$	Capas con tanh / sigmoide / softmax : mantiene varianza balanceada.
Uniforme	$\mathcal{U}[-0,1, +0,1]$	Perceptrón simple del Ej 1 (sin propagación profunda).

Optimizador — cuál y por qué

La clase introduce **gradiente descendente clásico** (SGD); Momentum y Adam son extensiones modernas. Comparamos los tres.

- **SGD vanilla:** $w \leftarrow w - \eta \cdot \nabla L$. Lo más simple. Funciona pero es lento, especialmente en superficies con valles alargados.
- **Momentum** ($\beta = 0,9$): acumula *velocidad* en la dirección del gradiente. Atraviesa valles sin oscilar.
- **Adam** ($\beta_1 = 0,9$, $\beta_2 = 0,999$, $\varepsilon = 10^{-8}$): combina momentum con *learning rate adaptativo por parámetro*. El PDF de optimizadores lo califica textual como “*muy usado en la práctica*” (defaults idénticos al paper Kingma & Ba 2015).

Resultado en Ej 2 (sweep Fase 1.2, K-fold=5):

Adam 96.74 % (best_ep=7) $\approx$ **Momentum** 96.34 % $\gg$ **SGD** 94.81 % (no convergió en 50 epochs).

Elegimos Adam por convergencia más rápida y resultado igual o mejor.

Learning rate — cuál y por qué

Recomendación de la clase: learning rate *pequeño*; probar varios órdenes de magnitud y mapear cómo afectan a la convergencia.

- **Demasiado grande** → la red oscila o diverge.
- **Demasiado chico** → no converge en tiempo razonable.

Experimentación en dos pasos (Ej 2):

Etapa	Valores barridos	Resultado
Fase 1 exploratoria (k=1)	$\{10^{-4}, 5 \cdot 10^{-4}, 10^{-3}, 5 \cdot 10^{-3}, 10^{-2}\}$	10^{-3} ganó
Fase 2 validación (k=5)	$\{10^{-4}, 10^{-3}, 10^{-2}, 0, 1, 10\}$ con extremos	10^{-3} confirmado

Elegimos $lr = 10^{-3}$ con Adam.

Learning rate — Fase 1 exploratoria (k=1)

LR	val_acc	macro_f1	best_ep	total_ep	Comportamiento
0.001	96.74 %	0.863	7	18	✓ Óptimo
0.0005	96.54 %	0.864	17	28	Lento
0.0001	96.22 %	0.861	47	50	No converge
0.005	96.02 %	0.857	4	15	Overshoot moderado
0.01	95.30 %	0.850	2	13	Overshoot fuerte

Tres criterios convergen en $lr = 10^{-3}$:

- Máximo claro de **val_acc** (forma de parábola).
- Convergencia rápida: **best_epoch=7** vs 47 con 10^{-4} .
- Sin overshoot ($lr \geq 5 \cdot 10^{-3}$ corta a las 2-4 épocas, demasiado temprano).

Learning rate — Fase 2 validación (k=5)

LR	val_acc	best_ep	Tiempo / Comportamiento
0.001	96.22 %	4.6	449 s
0.0001	95.79 %	31.6	1390 s (3,1× más lento)
0.01	94.71 %	4.6	455 s, overshoot
0.1	29.00 %	7.2	divergencia catastrófica
10	12.48 %	11.6	NaN, loss explota

Extremos catastróficos confirmados. Con $lr = 10$ la red queda en accuracy random (12 %, equivalente a 1/10 clases). Con $lr = 0,1$ también diverge.

$lr = 10^{-3}$ **confirmado por dos sweeps independientes.** Robusto frente a la varianza del K-fold.

Convergencia y bias — decisiones de implementación

Criterio de convergencia

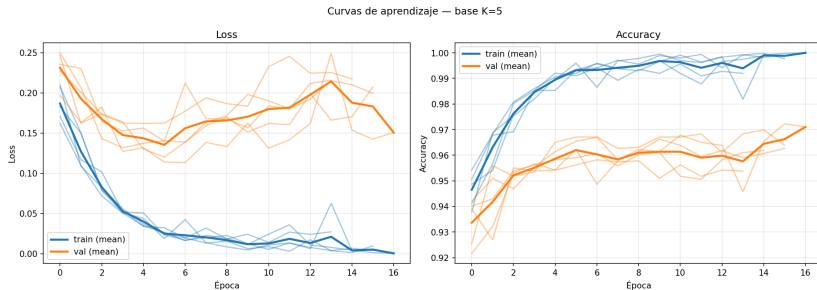
Ej	Criterio
Ej 1	ε absoluto sobre MSE de train (<i>regla del apunte</i> para perceptrón simple). Corta cuando $mse < \varepsilon$.
Ej 2/3	Early stopping sobre <code>val_loss</code> con <code>patience=10</code> .

Early stopping detecta overfitting que ε solo no detecta.

Manejo del bias

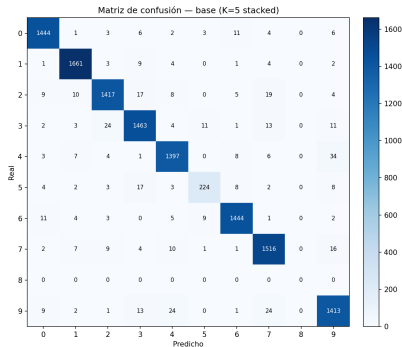
- **Bias trick:** agregamos columna de 1's al input de cada capa $\Rightarrow$ pesos con shape `(n_out, n_in+1)`. El bias es un peso más, se actualiza con la misma regla de gradiente.

Ej 2 — Curvas de aprendizaje del modelo base (K=5)



Convergencia limpia en ~ 5 epochs. Train y validación bajan juntos: **sin overfitting visible**.

Ej 2 — Matriz de confusión del base (val K=5)



Diagonal limpia para 9 de 10 clases. La clase 8 colapsa (precision = recall = 0). $\Rightarrow$ Hint: algo raro pasa con la distribución de los datos.

val K-fold=5: **96.22 %**

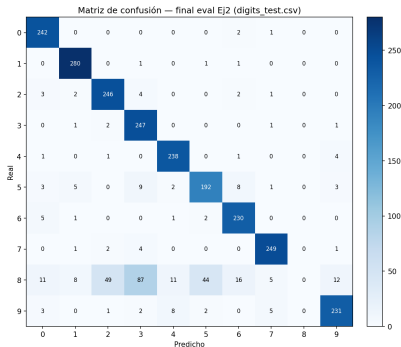


test (digits_test.csv): **86.30 %**

Drop de 10 puntos porcentuales.

Distribution shift, no underfitting. El modelo está bien — los datos no se parecen.

Ej 2 — Matriz de confusión sobre digits_test.csv



Shift, no falta de capacidad: confusiones distribuidas. $\Rightarrow$ **Más datos**, no más modelo. Eso es lo que el Ej 3 provee.

Bloque 3 / 3

Ejercicio 3 — Conclusiones

Ej 3 — Hipótesis y plan

Hipótesis: el techo del Ej 2 es por distribution shift, no por falta de capacidad.

Plan secuencial:

1. **Más datos.** Concatenar `more_digits.csv` (15.742 muestras adicionales). Si llega a $\geq 0,98$, listo.
2. **Se emplea regularización** (L2, dropout, augmentation gaussiana, lr scheduling) y combinaciones.
3. **Capacidad.** Si nada llega, probar `arch_wider`.

Ej 3 — El movimiento dominó

val $K=5$: 96,22 % $\rightarrow$ 97,06 %

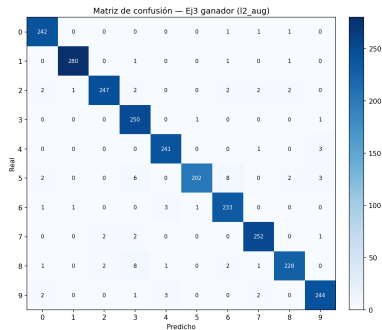
test: 86,30 % $\rightarrow$ 96,36 %

+10 puntos porcentuales en test.
Cero cambios al modelo. Solo más datos.

Macro F1: 0,857 $\rightarrow$ 0,959.

La clase 8 que estaba colapsada en Ej 2 ahora se predice correctamente. Más datos cubrieron el agujero de cobertura. Confirma la hipótesis del shift.

Ej 3 — Matriz de confusión del ganador (test)



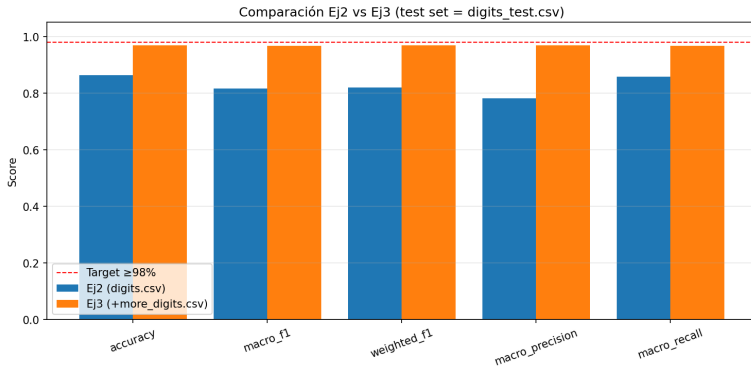
La clase 8 que estaba colapsada en Ej 2 ahora se predice correctamente. La diagonal está mucho más limpia, los errores residuales se reparten — síntoma de que ya no hay un agujero específico de cobertura, sino limitaciones globales del modelo.

Ej 3 — Resultados

Config	val K=5	test	Δ vs base_extra
base_extra_data	0.9706	0.9636	— (referencia)
+ L2 (1e-4)	0.9758	0.9656	+0,20 pp
+ dropout (p=0.2)	0.9750	0.9628	−0,08 pp
+ L2 + dropout	0.9772	0.9604	−0,32 pp
+ L2 + aug ($\sigma = 0,05$)	0.9760	0.9688	+0,52 pp
+ L2 + aug + dropout	0.9768	0.9656	+0,20 pp
+ wider + L2 + aug	0.9777	0.9640	+0,04 pp

Ganador: L2 + augmentation gaussiana ($\sigma = 0,05$) → **96.88 % en test.**

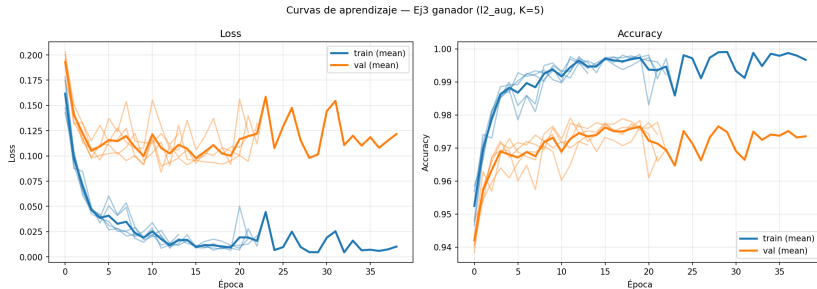
Ej 3 — Comparación visual con Ej 2



Línea roja: target $\geq 98\%$.

El salto principal viene del baseline con extra data, no de la regularización fina.

Ej 3 — Curvas de aprendizaje del modelo ganador



Convergencia más lenta que Ej 2 base ($\text{best_ep} \sim 10$ vs 5) por la regularización L2 + augmentation. Sin overfitting visible.

Ej 3 — Resultados: qué ayudó y qué no

Lo que ayudó:

- **Más datos: +10 pp.** El movimiento dominante.
- **L2: +0,20 pp** consistente.
- **Aug gaussiana: +0,32 pp** *adicional* sobre L2 (sinergia).

Lo que no ayudó:

- **Dropout:** gana en val, pierde en test. Reduce varianza al fold, no al shift.
- **wider** [784, 200, 100, 10]: mejor en val, peor en test. Descarta falta de capacidad.
- **Combinar todo:** L2+dropout+aug peor que L2+aug solo.

Ej 3 — Por qué no llegamos al 98 %

Mejor: 96.88 %. Faltan 1.12 pp.

Hipótesis principal:

- El **augmentation gaussiano** simula *ruido por píxel*. Pero el shift entre train y test parece ser *geométrico*: distintas personas escriben los dígitos con rotación, desplazamiento y grosor de trazo distintos.
- El **PDF de regularización lista cuatro formas de augmentation; sólo implementamos una** (gaussiano). Faltan rotación, traslación y cambios de escala.
- Aumentar la capacidad (**wider**) empeoró: **no es problema de modelo, es problema de cobertura**.

Próximo paso natural: augmentation geométrica (rotación / traslación pequeñas).

Conclusiones del TP

Tres ideas para llevarse:

1. **Una buena pipeline gana más que tunear hiperparámetros.**
El +10pp del Ej 3 vino de *datos*, no de modelo.
2. **Los empates técnicos son más comunes que los wins claros.**
Hay que elegir por **simplicidad**, tener criterio de complejidad en tiempo y espacio, no solo por pequeñas diferencias de resultados pp.
3. **“Mejor en val” $\neq$ “mejor en test” cuando hay shift.**
Dropout fue el ejemplo de manual: ganador en val, perdedor en test.

Item	Status
Ej 1 — Knowledge distillation + threshold sweep	✓
Ej 2 — sweeps + final_eval + análisis del shift	✓
Ej 3 — more_digits.csv + regularización + análisis	✓
Ej 3 — target $\geq$ 98 %	✗ (96.88 %)

Gracias.
